```
  if x == null or isRoot(x) then
                 #2
    throw

  y ←
x.parent
    #3
  throw-if y.left !=
x                                                    #4
  p    ← y.parent
                          #5
  if p != null then
                                  #6
    if p.left == y then
                          #7
      p.left ←
x                                                        #8
    else
      p.right ← x
  else
        #9
    treap.root ← x
  y.left ← x.right
                          #10
  x.right ←
y
#11
```

#1 Method `rightRotate` takes a `Treap` node `x` and performs a right rotation. It returns nothing but it has side effects on the treap.

#2 Check if `x` is null or the root of the tree: if it is so, there is an error (arguably, we can just return instead of throwing, but swallowing exceptions is usually not a good idea). Method isRoot is left to the reader to implement (easily, since the root is the only node in a tree without a parent…).

#3 Use a convenience variable for `x`'s parent. Since `x` is not the root of the tree, `y` won't be `null` (so, no need for an extra